
StudyFlow

CMP6213 MOBILE & WEARABLE APPLICATION DEVELOPMENT

Student Name: Nguyễn Võ Chí Dũng

Student Id: 25195662

Table of Contents

- 01 Introduction & Core App Features
- 02 Problem Statement & Motivation
- 03 Core Features
- 04 Deep Focus & Distraction Tracker
- 05 Performance Signals
- 06 System Architecture
- 07 Technology Stack
- 08 Feature Implementation
- 09 Database Design
- 10 UI Design & Theming
- 11 Testing Strategy
- 12 Discussion & Conclusion

01

Problem Statement & Motivation

Current Student Reality



Key Problems

Cognitive Fatigue

Switching between multiple apps drains mental energy and breaks focus flow

Privacy Risks

Personal health & biological logs stored on cloud servers without student control

Weak Offline

Existing tools require internet, failing students in libraries or transit

StudyFlow consolidates everything locally on-device: a private, offline-first workspace where productivity and wellness indicators converge.

02

Core Features

Workspace Compartments

Segment tasks, notes, habits by course, semester, or project with color-coding

Task Planner

Priority-based CRUD with swipe deletion, filter chips, and workspace mapping

Habit Streaks

Daily completion tracking with automatic streak calculation and visual heatmaps

Pomodoro Focus

Breathing halos, distraction counter, auto-tracked app exits, noise synthesis

Notes & Auto-Save

Subject-specific notebooks with 1-second debounce auto-save via coroutines

Analytics & Timeline

Vico charts, productivity correlation, chronological activity feed

All features unified under a single Dashboard with workspace-based filtering and offline-first data persistence.

03

Deep Focus: Breathing Halos & Exit Tracking

Pomodoro Timer Engine

- Circular sweep progress indicator
- Breathing pulse halos (16-second cycle) expand/contract to guide user focus
- White & Brown noise synthesis generated natively via AudioTrack API
- Configurable durations:
Focus | Short Break | Long Break
- Task binding: link session to specific task label and workspace

Auto Distraction Tracking

- App-exit counter increments automatically when user backgrounds the app during focus
- Logs phone pickups & off-track moments
- Data saved to PomodoroSessionEntity

Session End Reflection

- Focus rating (1-5 scale)
- Reflection note for session summary
- Distraction count displayed
- All data persisted to Room database

Screenshots: Active Pomodoro Timer (left) | Reflection Dialog with App Exits count (right)

04

Performance Signals: Biological Correlation

Wellness Metrics

Sleep 7-9 hrs = Positive impact

Caffeine Over-consumption = Negative

Energy Scale 1-5, daily tracking

Stress Scale 1-5, correlates with focus

Hydration ml intake logged daily

Screenshots: Add Signal Sheet (left) | Weekly Activity Bar Chart (right)

Auto-Calculated Impact

The system automatically calculates productivity impact of each log in real-time:

- Sleep 7-9 hrs → flagged as Positive
- Caffeine > threshold → flagged as Negative
- Color-coded chips update dynamically
- Impact stored as: -1 (Neg) | 0 (Neutral) | +1 (Pos)

Environment tagging: Home, Library, Cafe, School

Study Environment Correlation

Location tagging correlates environmental choice with productivity output (Canvas bar charts)

04

System Architecture & Design



Key Design Decisions

- MVVM + Repository pattern
- Single Activity architecture
- Hilt for dependency injection
- Sealed class navigation routes

Project Structure

```
app/  
  data/ (entities, dao, repository)  
  di/ (Hilt modules)  
  ui/ (feature screens)  
  navigation/ (routes)  
  theme/ (dynamic theming)
```

05

Technology Stack

Core

- Kotlin (100%)
- Jetpack Compose
- Material Design 3
- MVVM Architecture
- Android API 26-36

Key Libraries

- Hilt (Dependency Injection)
- Room (Local Database)
- Navigation Compose
- Kotlin Coroutines & Flow
- DataStore (Preferences)

UI & Analytics

- Vico (Charts & Graphs)
- Material Icons Extended
- Accompanist libraries
- WorkManager
- Kotlin Serialization

Build System

Gradle with Kotlin DSL | Version Catalog (libs.versions.toml) | KSP for annotation processing | Single-module structure

06

Feature Implementation

Dashboard

Card-based hub, workspace filter, habit toggles, focus carousel

Task Management

Filter chips, swipe-to-dismiss, priority sliders, workspace mapping

Habit Tracking

Interactive checklist, streak evaluation, daily epoch logging

Pomodoro Timer

Configurable modes, distraction counter, task binding, reflection

Notes

Subject categorization, auto-save with coroutine debounce

Signals Tracker

Caffeine, sleep, stress, energy, hydration & environment logging

Analytics

Vico charts, heatmaps, category breakdowns

Timeline

Chronological feed of tasks, sessions & reflections

Settings & Theme

Dark mode, danger zones, confirmation dialogs

07

Database Design

Room (SQLite) | 9 Entities | 8 DAOs | Repository Pattern

WorkspaceEntity

Courses/projects with
name, emoji, color

TaskEntity

Deliverables with priority,
due date, workspace FK

HabitEntity

Daily routines with
current & best streak

HabitLogEntity

Individual completions
with habitId FK, dateMillis

NoteEntity

Subject docs with
title, content, isPinned

PomodoroSessionEntity

Focus logs with duration,
rating, distractions

ResourceEntity

Signals: caffeine, sleep,
stress, environment

ReflectionEntity

Daily review: mood,
accomplishments, stress

UserSettingsEntity

Single-row config:
theme, timer durations

All entities use Long primary keys with auto-generate. Foreign keys link to Workspaces (1:N). CASCADE on delete for HabitLog.

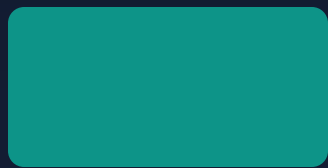
08

UI Design & Theming

Brand Colors



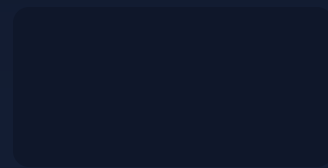
Primary
Deep Indigo



Secondary
Emerald Teal



Tertiary
Amber Gold



Surface
Dark Navy

Dynamic Theme System

Three reactive parameters control the entire UI:

- `isDarkTheme` - Light/Dark mode toggle
- `isRecoveryMode` - Reduced visual intensity for breaks
- `currentStudyState` - FOCUS / SHORT_BREAK / LONG_BREAK

All changes propagate instantly via Compose recomposition.

Key Custom Components

`StudyFlowTopBar`

Workspace color, theme selectors, contextual actions

`PriorityChip`

Color-coded High/Medium/Low task indicators

`EmptyStateView`

Themed illustrations with supportive messages

`ConfirmDialog`

Safety utility for destructive actions

Built entirely on Jetpack Compose Material 3 components (Scaffold, NavigationBar, TopAppBar, Cards, Chips)

09

Testing Strategy

Database Unit Tests

- JUnit 4 + AndroidJUnit4 runner
- In-memory Room database instances
- Verify CRUD operations on all DAOs
- Test Flow emissions on data changes
- Isolated from production database

```
@Test
fun writeHabitAndReadInList() {
    dao.insertHabit(habit)
    assertEquals(habits[0].name, expected)
}
```

Domain Logic Tests

- Streak calculation verification
 - Consecutive days increment streak
 - Skipped days reset to 1
 - Multi-day gaps handled correctly
- Pomodoro state transitions
 - FOCUS -> SHORT_BREAK
 - SHORT_BREAK -> FOCUS
 - After 4 cycles -> LONG_BREAK
- Repository business rules

UI & Integration Tests

- Compose Testing (createComposeRule)
- Verify checkbox toggle animations
- Slider value synchronization
- Deletion confirmation dialog flow
- Navigation between screens
- Empty state view rendering
- Theme switching reactivity

All tests run locally without network dependencies. No cloud components required. Fake repositories and in-memory databases ensure fast, reliable test execution.

10

Discussion & Conclusion

Technical Challenges

1. Timer Persistence
Pomodoro survives process recreation
via ViewModel Coroutine flows
2. Date Boundaries
Epoch stamps at 00:00:00 for timezone safety
3. Responsive Themes
Dynamic color schemes with local
settings observation in Compose

Future Work

- Health Connect API integration
(automated sleep/wellness tracking)
- Academic Export
(PDF & Markdown for notes/stats)
- Distributed Synchronization
(E2E encrypted SQLite replication
via WebDAV for multi-device sync)

Conclusion

StudyFlow delivers a unified, high-performance, and secure productivity suite for students.

Built with Kotlin, Jetpack Compose, Room, and Hilt under Clean Architecture - the codebase is maintainable, stable, and highly testable.

Studying becomes organized and distraction-free.

Key Takeaway: StudyFlow uniquely converges wellness tracking, task management, and focus sessions in a private, offline-first Android ecosystem.

Thank You

Questions & Discussion